# Tools — MCP

Bridge to external Model Context Protocol (MCP) servers with auto-discovery of known server types and stdio clients.

A server entry's `"type"` is optional and defaults to `stdio`, so a command-launched server needs only `command` and `args`; the same default applies to plugin `mcpServers` bridges and connector `servers` / `remote` entries, and a config written back still names its transport explicitly.

| Tool | Description |
|---|---|
| `mcp_tool` | Bridge to external Model Context Protocol servers. |

MCP server tools surface dynamically as `mcp_<server>_<tool>` at runtime after discovery via the `/mcp discover` TUI command, or after `/mcp connect <id>` enables and connects a server live. Servers bridged from an enabled plugin's `mcpServers` section connect as `<plugin-id>.<server>` at startup.

The `mcp_` namespace is what keeps a server out of the bare ragent namespace: a server that offers a tool called `bash` gets `mcp_<server>_bash`, which is not the retired `bash` and cannot make it resolvable. A bridge whose surfaced name falls outside that namespace, or that equals a name this build retired, is refused with a message naming the server, the spelling and the name to use instead; the rest of that server's tools keep working, and the refusal is reported once rather than on every reconnect. Connector servers obey the same rule - they reach the registry through this bridge under a dotted id.

---

## mcp_tool

The `McpToolWrapper` calls a tool hosted on a connected MCP server.

**Arguments**

| Argument | Type | Required | Description | Typical value |
|---|---|---|---|---|
| `server` | string | yes | MCP server name (as config-ured/discovered) | `"filesystem"` |
| `tool` | string | yes | Tool name exposed by the server | `"read_file"` |
| `arguments` | object | no | JSON payload forwarded to the server tool | `{"path":"/tmp/x"}` |

**Example:**

```
mcp_tool server="github" tool="create_issue" arguments={"title":"Bug report"}
```

TUI commands: `/mcp` / `/mcp status` (server table), `/mcp discover` (probe for known servers), `/mcp connect <id>` and `/mcp disconnect <id>` (enable/disable and connect/disconnect live, persisted globally in `<state dir>/mcp_state.json`). The wrapper refuses to call a disabled server's tools even when the tool is still registered, and names the command that re-enables it.

---

## Connection lifecycle

Every `McpClient::connect` attempt runs under a bounded per-attempt timeout (default 30 s, overridable with `RAGENT_MCP_CONNECT_TIMEOUT_SECS`) and is retried once on timeout only. A launcher such as `npx`/`npm exec` that stalls resolving a package, or a server that accepts the connection and then never answers the handshake, fails within the bound with an actionable error instead of hanging startup; a timed-out stdio attempt has its partially-started child torn down before the retry so the two attempts cannot collide. Genuine errors are surfaced immediately without a retry.

Startup does not block on the connect loop. The TUI adopts whatever state the loop has published and prints its one-shot per-server report off the loop's completion sentinel, an empty-id `initialized` event:

```
[mcp] Starting <id> (<transport>)
[mcp] Connected to mcp server <id> via <transport>
[mcp] Failed to connect to mcp server <id> via <transport>: <reason>
```

The report ends with one blank line, separating it visually from whatever the stream prints next.

Every connect failure path records the server as `McpStatus::Failed { error }` before returning, so a server rejected by config validation (e.g. a stdio entry with no `command`) or otherwise failing before a transport is attempted is reported here and listed by `/mcp` with its reason, rather than only a single `warn` in the log panel. The startup connect loop publishes this client-derived status (`connected`, else `failed: <reason>`) on its per-server `McpStatusChanged` event, so `/mcp` shows the reason too.